

การติดตั้ง PREEMPT_RT Kernel บน Compute Module 5 (CM5)

1. โครงสร้างฮาร์ดแวร์ Raspberry Pi CM5 ที่เกี่ยวข้อง

การคอมไพล์ Real-Time Kernel บน Compute Module 5 (CM5)

มีข้อได้เปรียบสูงเนื่องจากสถาปัตยกรรมฮาร์ดแวร์ระดับอุตสาหกรรม

ที่ทรงประสิทธิภาพ: หน่วยประมวลผลหลัก (CPU): Broadcom BCM2712 (Quad-core ARM Cortex-A76) ความเร็ว

2.4GHz รองรับสถาปัตยกรรมแบบ 64-bit (aarch64) มีความเร็วเทียบเท่าคอมพิวเตอร์ระดับเดสก์ท็อป

หน่วยความจำ (RAM): รองรับสูงสุด 16GB LPDDR3 ช่วยให้มีพื้นที่ Buffer และ Cache

เพียงพอในการรันคำสั่งคอมไพล์แบบขนานเต็ม 4 คอร์ (make -j4)

ข้อจำกัดฮาร์ดแวร์: การทำ Real-Time Kernel จำเป็นต้องใช้ Source Code เฉพาะของทางคลัง Raspberry Pi (ไม่ใช่โครงสร้างหลักของ kernel.org) เพื่อรักษาระบบไดรเวอร์ควบคุมฮาร์ดแวร์และพอร์ตอินเทอร์เฟซเฉพาะของ CM5 ให้ทำงานร่วมกันได้อย่างสมบูรณ์

2. ขั้นตอนการเตรียมระบบและการติดตั้ง Dependencies

ก่อนเริ่มกระบวนการจัดเตรียมซอร์สโค้ด จำเป็นต้องติดตั้งเครื่องมือและแพ็คเกจพื้นฐานที่จำเป็นสำหรับกระบวนการคอมไพล์

(Build Essentials):

Bash

```
sudo apt update
```

```
sudo apt install -y build-essential bison flex libssl-dev libelf-dev libncurses-dev bc git wget xz-utils
```

3. การจัดการซอร์สโค้ดและการกำหนดค่าผ่าน Menuconfig

เนื่องจาก Compute Module 5 (CM5) ต้องการไดรเวอร์ฮาร์ดแวร์ที่จำเพาะเจาะจง ทางผู้พัฒนา Raspberry Pi จึงได้ทำการรวบรวม (Integrate) ซอร์สโค้ดและ Real-Time Patch ไว้ในคลัง GitHub หลักอย่างเป็นทางการ ผู้พัฒนาไม่จำเป็นต้องแยกดาวน์โหลดไฟล์ Patch (.patch.xz) มาติดตั้งเอง ซึ่งช่วยลดความเสี่ยงจากการเกิดข้อผิดพลาดประเภท Reversed patch

3.1 การดึงซอร์สโค้ดจากคลัง (Cloning Source Code)

ทำการดึงข้อมูลซอร์สโค้ดตามเวอร์ชันของ Kernel ปัจจุบัน (ตัวอย่างอ้างอิงเวอร์ชันย่อย 6.18.y):

Bash

```
mkdir ~/pi_rt_kernel && cd ~/pi_rt_kernel  
git clone --depth=1 --branch rpi-6.18.y https://github.com/raspberrypi/linux.git  
cd ~/pi_rt_kernel/linux
```

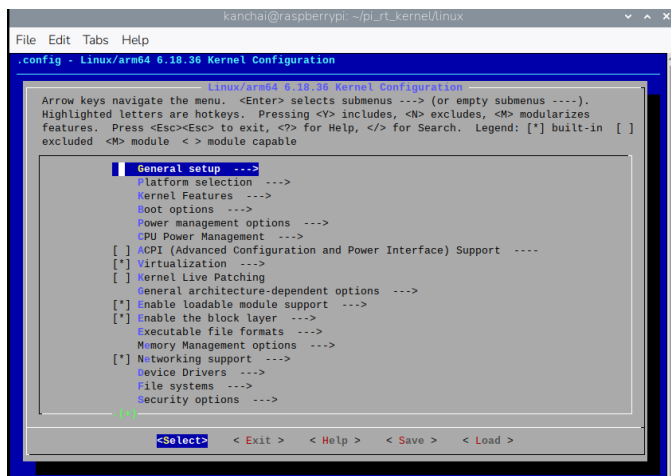
3.2 การสร้าง Configuration พื้นฐานและการปรับแต่งผ่านอินเทอร์เฟซระบบ

สร้างไฟล์กำหนดค่าสถาปัตยกรรมสำหรับชิป BCM2712 บน Compute Module 5 (CM5):

Bash

```
make bcm2712_defconfig  
make menuconfig
```

ตัวอย่างหน้า UI



เมื่อเข้าสู่หน้าจอเมนูกราฟิกสีน้ำเงิน-เทา (Menuconfig Interface)ดำเนินการตั้งค่าเฉพาะเจาะจงเพื่อเปิดระบบ Real-Time ดังต่อไปนี้:

หัวข้อหลัก (Main Menu), เมนูกลุ่มย่อย (Sub-menu), พารามิเตอร์ที่ต้องแก้ไข (Parameter),

วัตถุประสงค์ (Purpose)

General Setup-> Preemption Model, เปลี่ยนเป็น Fully Preemptible Kernel (Real-Time),

เปิดใช้งานกลไกขัดจังหวะการทำงานในระดับโครงสร้างหลักของ Kernel เพื่อลด Latency โดยการเปิดแบบนี้ [*] Fully Preemptible Kernel (Real-Time)

General Setup, Timers subsystem, เปิดใช้งาน [*] High Resolution Timer Support,

รองรับสัญญาณนาฬิกาที่มีความแม่นยำสูงระดับไมโครวินาทีสำหรับงาน Real-Time

CPU Power Management ->CPU Frequency scaling->Default CPUFreq

governorเปลี่ยนเป็น (x)performance, บังคับให้ CPU

วิ่งที่ความเร็วสูงสุดตลอดเวลาป้องกันความหน่วงจากการปรับความถี่ตามภาระงาน

Kernel hacking-> Compile-time checks and Compiler options-> Debug information

ติ๊ก (x) Disable debug information, ปิดการสร้างไฟล์สัญลักษณ์ตรวจสอบ บั๊ก ช่วยลดขนาดไฟล์ Kernel ลง 10 เท่า

และเพิ่มความเร็วในการคอมไพล์ กดเลื่อนไปที่ < Save > ด้านล่าง แล้วกด Enter เพื่อบันทึกไฟล์ .config กด < Exit >

ออกมา แล้วเริ่มทำการคอมไพล์และการติดตั้ง

4.กระบวนการคอมไพล์และการติดตั้งระบบ (Compilation & Installation)

เมื่อสิ้นสุดการบันทึกค่าลงในไฟล์ .config

ให้ดำเนินการคอมไพล์ซอร์สโค้ดโดยใช้หน่วยประมวลผลทั้ง 4 คอร์ของ Compute Module 5 (CM5):

Bash

```
make -j4 Image modules dtbs
```

หลังจากกระบวนการคอมไพล์เสร็จสิ้นโดยไม่มีข้อผิดพลาด ให้ทำการติดตั้ง Modules

และย้ายระบบไฟล์อิมเมจไปยังพาร์ติชันระบบบูตหลัก (Boot Firmware Partition) ของเครื่อง:

Bash

```
sudo make modules_install
```

```
sudo cp arch/arm64/boot/Image /boot/firmware/kernel8-rt.img
```

```
sudo cp arch/arm64/boot/dts/broadcom/*.dtb /boot/firmware/
```

```
sudo cp arch/arm64/boot/dts/overlays/*.dtbo /boot/firmware/overlays/
```

```
sudo cp arch/arm64/boot/dts/overlays/README /boot/firmware/overlays/
```

4.1 การกำหนดสิทธิ์การบูต (Boot Configuration)

ทำการเปิดไฟล์กำหนดค่าการบูตระบบของ Raspberry Pi:

Bash

```
sudo nano /boot/firmware/config.txt
```

เพิ่มคำสั่งเงื่อนไขโครงสร้างไว้ที่ส่วนท้ายสุด เพื่อกำหนดให้บอร์ดเรียกใช้งานอิมเมจ Real-Time

Kernel ตัวใหม่ที่สร้างขึ้นทดแทนระบบเดิม:

[all]

```
kernel=kernel8-rt.img
```

```
arm_boost=1

[cm4]
# Enable host mode on the 2711 built-in XHCI
# This line should be removed if the legacy
# (e.g. for USB device mode) or if USB supp
otg_mode=1

[cm5]
dtoverlay=dwc2,dr_mode=host

[pi5]
dtoverlay=nospi10

[all]

kernel=kernel8-rt.img
```

(ถ้าอยากกลับมาใช้ PREEMPT เดิมให้ใส่ # แบบนี้ #kernel=kernel8-rt.img)

บันทึกไฟล์กด ctrl + o แล้วกด Enter

แล้วกด ctrl + x เพื่อ ออก แล้วทำการreboot* ทำการบันทึกไฟล์และรันคำสั่งรีบูตระบบ: sudo reboot

5.การทดสอบและการวัดผลประสิทธิภาพ (Benchmarking)

เมื่อระบบเปิดใช้งานขึ้นมาใหม่ สามารถดำเนินการตรวจสอบสถานะความเป็น Real-Time

ผ่านคำสั่งระบบ:

Bash

```
uname -a
```

ผลลัพธ์ที่ถูกต้องควรปรากฏคำว่า PREEMPT_RT หนาข้างเวอร์ชันระบบ

```
cat /sys/kernel/realtime
```

ผลลัพธ์ที่แสดงต้องส่งกลับค่าเป็นเลข "1" เพื่อยืนยันสถานะการเปิดใช้งานสมบูรณ์

6. ขั้นตอนการตั้งค่าระบบเพื่อทำการสลักแกนประมวลผล (System Configuration)

การลือกคอร์สามารถทำได้โดยการส่งพารามิเตอร์ผ่าน Command Line ของ Kernel (Kernel boot parameters)

ในตอนทีระบบเริ่มบูต โดยมีขั้นตอนปฏิบัติดังนี้:

6.1 การตั้งค่าพารามิเตอร์ผ่านไฟล์ระบบ

ทำการปรับแต่งไฟล์ cmdline.txt ซึ่งเป็นไฟล์กำหนดค่าการเริ่มต้นระบบของ Raspberry Pi:

Bash

```
sudo nano /boot/firmware/cmdline.txt
```

ให้ทำการเพิ่มชุดคำสั่งต่อไปนี้ต่อท้ายบรรทัดเดิม (โดยเว้นวรรค 1 ช่องระหว่างคำสั่ง และห้ามขึ้นบรรทัดใหม่):

Plaintext

```
isolcpus=2,3 nohz_full=2,3 rcu_nocbs=2,3
```

6.2 คำอธิบายฟังก์ชันการทำงานของพารามิเตอร์ (Parameter Dissection)

isolcpus=2,3 (Isolate CPUs): สั่งการให้ตัวจัดตารางงานหลัก (Standard Linux Scheduler) ลบคอร์ 2 และ 3

ออกจากการคำนวณสิทธิ์กระจายงาน ส่งผลให้กระบวนการทำงานทั่วไป (General Processes)

ของระบบปฏิบัติการจะไม่สามารถเข้ามาทำงานบนคอร์นี้ได้โดยอัตโนมัติ

nohz_full=2,3 (Adaptive-tick system): กำหนดให้คอร์ 2 และ 3 ทำงานในโหมด "Tickless"

หรือลดการขัดจังหวะจากสัญญาณนาฬิกาภายในระบบ (System Clock Interrupts)

ให้เหลือน้อยที่สุดเมื่อมีแอปพลิเคชันทำงานอยู่ ช่วยลด Overhead และ Latency Jitter

rcu_nocbs=2,3 (RCU Offloading): ทำการย้ายงานเบื้องหลังของระบบควบคุมหน่วยความจำ (Read-Copy Update callbacks) ออกไปจากคอร์ที่เลือก เพื่อไม่ให้ส่งผลกระทบต่อประสิทธิภาพการประมวลผลแบบเรียลไทม์

7. การกำหนดทิศทางการรันโค้ดของผู้ใช้ (Binding User Application to Isolated CPUs)

เมื่อระบบถูกบูตขึ้นมาใหม่ คอร์ 2 และ 3 จะอยู่ในสถานะถูกล็อกและว่างสนิท การจะส่ง Real-time Code ของผู้ใช้ไปรันบนคอร์ดังกล่าว สามารถทำได้ 2 ระดับ:

7.1 ระดับคำสั่งควบคุมระบบ (System Command Level)

ใช้คำสั่ง taskset ควบคู่กับรหัสสิทธิ์การเข้าถึงแกน (CPU Affinity Mask) เพื่อสั่งเปิดโปรแกรมประยุกต์บนคอร์ที่กำหนด:

Bash

```
sudo taskset -c 2,3 ./my_realtime_program
```

7.2 ระดับการเขียนโปรแกรมควบคุม (Source Code Level - C/C++)

หากต้องการระบุลงไปในระดับซอร์สโค้ดโปรแกรม เพื่อให้โปรแกรมเรียลไทม์ล็อกคอร์ด้วยตัวเองเมื่อเริ่มทำงานสามารถใช้ฟังก์ชันในไลบรารี <sched.h> ของระบบ POSIX ได้ดังนี้:

ภาษา C

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <unistd.h>

int main() {
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);    // ล้างค่าชุดข้อมูล CPU
    CPU_SET(2, &cpuset);   // บังคับเลือก Core 2
    CPU_SET(3, &cpuset);   // บังคับเลือก Core 3

    // สั่งการให้เธรด (Thread) ปัจจุบัน ผูกขาดการทำงานไว้ที่ Core 2 และ 3 เท่านั้น
    if (sched_setaffinity(0, sizeof(cpu_set_t), &cpuset) < 0) {
        perror("sched_setaffinity failed");
        return 1;
    }

    printf("Real-time code is now locked and running on Core 2 & 3\n");

    /* ใส่โค้ดหรือฟังก์ชันงานที่เป็น Real-time Code ตรงนี้ */
    return 0;
}
```

7.3 ระดับการเขียนโปรแกรมควบคุมด้วยภาษา Python (Source Code Level - Python)

ในภาษา Python เราสามารถควบคุมสิทธิ์การเข้าถึงแกนประมวลผล (CPU Affinity)

ของโปรเซสปัจจุบันหรือเธรดได้โดยใช้ฟังก์ชัน `os.sched_setaffinity()`

ซึ่งเป็นไลบรารีมาตรฐานที่ติดต่อกับระบบปฏิบัติการลินุกซ์โดยตรง โครงสร้างโค้ดตัวอย่างมีดังนี้:

Python

```
import os
```

```
import time
```

```
def main():
```

```
    # 1. กำหนดหมายเลขคอร์ที่ต้องการล็อก (Core 2 และ Core 3)
```

```
    #หมายเหตุ: รันบนโปรเซสปัจจุบัน (PID = 0)
```

```
    target_cores = {2, 3}
```

```
    try:
```

```
        # 2. ล็อกโปรแกรมให้ทำงานเฉพาะบนคอร์ที่กำหนดเท่านั้น
```

```
        os.sched_setaffinity(0, target_cores)
```

```
        print(f"Successfully locked this Python script to Cores: {target_cores}")
```

```
    except AttributeError:
```

```
        print("CPU Affinity is only supported on Linux/Unix systems.")
```

```
        return
```

```
    except PermissionError:
```

```
        print("Error: You need root privileges (sudo) to set CPU affinity.")
```

```
        return
```

```
    # 3. ตรวจสอบสถานะเพื่อยืนยันว่าโปรแกรมถูกผูกเข้ากับคอร์อย่างถูกต้อง
```

```
    current_affinity = os.sched_getaffinity(0)
```

```
    print(f"Current running CPU Affinity: {current_affinity}")
```

```
    print("-----")
```

```
    print("Starting Real-time user loop on Core 2 & 3...")
```

```
    print("-----")
```

```
    # 4. ส่วนของ Real-time Code ของผู้ใช้ (ตัวอย่างเช่น ลูปควบคุมความถี่สูง)
```

```
    try:
```

```
        while True:
```

```
            # ใส่โค้ดคำนวณ หรือโค้ดควบคุมฮาร์ดแวร์เวลาจริงตรงนี้
```

```
            pass
```

```
except KeyboardInterrupt:
    print("\nProgram stopped by user.")

if __name__ == "__main__":
    main()
```

ข้อควรระวังในการรันบนรายงาน:

สิทธิ์ในการเข้าถึง (Root Privileges): การเรียกใช้งานฟังก์ชันจัดการฮาร์ดแวร์และคอร์ในระดับต่ำผ่าน Python

จำเป็นต้องรันสคริปต์ด้วยสิทธิ์ Root เสมอ โดยใช้คำสั่ง:

Bash

```
sudo python3 my_rt_program.py
```

Global Interpreter Lock (GIL): ในกรณีที่ใช้ Python มาตรฐาน (CPython) หากเขียนโปรแกรมแบบหลายเธรด (Multithreading) ตัว GIL จะบังคับให้ทำงานได้ที่ละ 1 เธรดสลับกันไป หากต้องการดึงประสิทธิภาพของทั้ง Core 2 และ 3 ออกมาพร้อมกันอย่างเต็มที่ในงาน Real-time แนะนำให้ใช้โครงสร้างแบบแยกโปรเซส (multiprocessing) แล้วแยกสั่ง `os.sched_setaffinity()` เจาะจงลงไปทีละคอร์ครับ (เช่น Process A ล็อกคอร์ 2, Process B ล็อกคอร์ 3)